



PDF Download
3708657.3708753.pdf
20 January 2026
Total Citations: 0
Total Downloads: 175

Latest updates: <https://dl.acm.org/doi/10.1145/3708657.3708753>

RESEARCH-ARTICLE

Programming Games for Fundamental Computational Concepts: A Technical Design Framework

BAIJUN CHEN, Beijing University of Posts and Telecommunications, Beijing, Beijing, China

Open Access Support provided by:

Beijing University of Posts and Telecommunications

Published: 21 November 2024

[Citation in BibTeX format](#)

ICCIP 2024: 2024 the 10th International
Conference on Communication and
Information Processing
November 21 - 24, 2024
Hainan, Lingshui, China

Programming Games for Fundamental Computational Concepts: A Technical Design Framework

Baijun Chen

School of Digital Media and Design Arts
Beijing University of Posts and Telecommunications
Beijing, China
chenbaijun@bupt.edu.cn

Abstract

With the rapid growth of artificial intelligence, the demand for programming education has surged, yet many young learners struggle with motivation due to the abstract nature of computational concepts. Games, with their engaging and contextual learning features, offer a promising solution. However, a large number of programming games fail to gain widespread adoption because of a disconnect between educational objectives and gameplay, often prioritizing the former at the expense of player engagement. This results in games that are perceived as tedious and lack immersion. To bridge this gap, we propose a framework that integrates game design principles from game flow theory, balancing educational content with engaging gameplay. We illustrate this framework with *WannaBone*, a programming game that introduces fundamental computational concepts such as loops, conditionals, and public boolean variables. Empirical research with primary school students shows that *WannaBone* enhances computational thinking, motivation, and game flow. This study provides both theoretical insights and practical guidance for the development of programming games, emphasizing the need to balance educational goals with compelling gameplay.

CCS Concepts

• **Human-centered computing** → Interaction design; Empirical studies in interaction design; • **Applied computing** → Education; Interactive learning environments; • **Social and professional topics** → Professional topics; Computing education; Student assessment; Professional topics; Computing education; K-12 education.

Keywords

Programming games, Fundamental computational concepts, Computational thinking, Design framework, Core game elements, Learning motivation, Flow experience

ACM Reference Format:

Baijun Chen. 2024. Programming Games for Fundamental Computational Concepts: A Technical Design Framework. In *2024 the 10th International*

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICCIP 2024, Lingshui, China

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1744-4/24/11

<https://doi.org/10.1145/3708657.3708753>

Conference on Communication and Information Processing (ICCIP 2024), November 21–24, 2024, Lingshui, China. ACM, New York, NY, USA, 8 pages.
<https://doi.org/10.1145/3708657.3708753>

1 Introduction

With the rapid advancement of artificial intelligence technology, numerous countries have placed great emphasis on computer science education. Programming equips individuals with essential skills such as critical thinking, problem-solving and information literacy which are crucial for the development of 21st century skills [1]. However, traditional programming education often faces challenges, such as students' lack of motivation and high absenteeism [2], as many students find programming tedious and daunting [3]. Particularly during their initial exposure to programming, students frequently experience frustration and monotony, leading to interruptions in their programming learning journey [4].

Learning activities that are active, contextual, problem-based, and provide immediate feedback yield higher student engagement and efficiency [5]. Games inherently embody these characteristics, making them a widely acknowledged educational tool [6]. However, the design of programming games presents considerable challenges, particularly in balancing educational objectives with gameplay [7]. Currently, programming games have not achieved widespread acceptance among teachers and students. While most existing programming games lack components that effectively create enjoyable experiences and enhance gameplay [8]. Many designers rely on their extensive experience with traditional teaching materials, placing greater emphasis on educational content while overlooking other elements that directly influence gameplay [9]. However, merely prioritizing educational content at the expense of gameplay is insufficient for designing a successful programming game.

Core game elements are crucial for balancing educational content and engaging gameplay in programming games. However, most existing research focuses primarily on the educational perspective when designing these core elements. This study aims to fill this gap by developing a design framework for the core elements of programming games, grounded in game flow theory and core gameplay processes. By doing so, it seeks to supplement current research and provide both theoretical support and empirical evidence to guide future programming game design.

2 RELATED WORK

2.1 Programming Games

Programming games are typically defined based on the concept of educational games, as games designed not only to create an

entertaining experience but also to include explicit educational objectives [10]. The primary functions of programming games are to assist students in better understanding computational concepts [11], familiarizing themselves with programming languages [12], developing computational thinking(CT) skills [13], while also enhancing their interest in programming and fostering a positive attitude towards the subject [14].

The primary goal of most programming games is to assist players in understanding abstract computational concepts through concrete game elements and scenarios [1, 15]. Programming games primarily encompass 5 categories of programming knowledge: algorithm design, fundamental programming concepts, fundamental data structures, development methods, and software design [11].

2.2 Core Elements in Programming Games

Researchers in academia hold diverse views on the elements that comprise a game. Adams categorizes game elements into challenge goals, play, rules, pretended reality [16]. Werbach and Hunter propose that games are constructed from components, mechanics, and dynamics [17]. Juul asserts that games consist of six elements: fixed rules, variable and quantifiable outcome, valorization of the outcome, player effort, attachment of the player to the outcome and negotiable consequences [18]. Debus classifies game elements as mechanics, spaces, time, entities, goals and randomness [19]. Sillaots et al. conducted a review of existing academic research and identified 138 types of game elements [20]. Based on related works and extending the research of Salen et al., we define the core elements of programming games as game mechanics, challenges, rules, and goals [21].

Game mechanics refer to the methods through which players or other entities engage with the game world [22]. Game mechanics can also be viewed as actions or functions—viable or encouraged methods that players can utilize during interactions with game characters to achieve their goals [23]. During the gaming, players continually utilize game mechanics to steer the game world toward their desired state. The game, in turn, presents a series of challenges to impede players from achieving their goals. The sense of success and frustration while overcoming challenges constitutes the fundamental source of gameplay [21]. Game rules consist of operational rules and constitutive rules, operational rules dictate how players interact with the game world, constitutive rules represent the fundamental structural form of the game, comprising a set of abstract logical relationships and numerical models [21].

2.3 Existing Design Frameworks for Programming Games

Although there has been some academic research focusing on the design framework and creative practice of programming games, existing studies are still insufficient to clarify how to create a gaming experience while achieving educational objectives.

Shabalina et al. proposed a framework for integrating programming and gaming. This framework is based on traditional programming course content, lack the view of gameplay [24]. Kazimoglu et al. proposed a macro-level model that describes the developing of CT through player-game interaction [13]. However, this framework primarily reflects abstract, process-oriented information and does

not illustrate the specific integration of programming with gaming. Malliarakis et al. developed the CMX framework including learning elements and game elements [25]. However, it does not elucidate the relationship and mutual influence between learning and gaming. Xina et al. proposed a framework for the core elements of games aimed at developing CT [26]. It primarily adopts an educational perspective and does not address how to create gaming experiences based on these core game elements.

To address the current gaps in research, our aim is to construct a framework for the core elements of programming games, not only clarifies how to achieve educational objectives but also how to create a gaming experience.

3 Framework and WannaBone Game

3.1 Framework for Core Elements in Programming Games

Programming is a problem-solving process that encompasses problem definition, algorithm design, and the validation of algorithm effectiveness during execution [27]. The programming process can be decomposed into problem formulation, algorithm building, and testing/debugging [28], while we can further divide it into four stages: problem identification and decomposition, data abstraction and representation, problem solving and evaluation, learning transfer and reflection.

Based on the concept of core game elements, we decompose the gaming process into four steps. Firstly, players clarify the game goals through exploring the game environment. Secondly, players develop strategies to achieve these goals by integrating constitutive rules and game mechanics. Thirdly, players utilize game mechanics through operational rules, while the game environment provides feedback based on constitutive rules. Finally, players continuously adjust their problem-solving strategies based on their observations of the game environment's state, ultimately guiding it towards the desired outcome.

Integrating programming into gaming process involves four key steps: 1) clarify the game goals by exploring the concrete game environment and identify the computational problems; 2) construct computational problem-solving strategies according to the constitutive rules and game mechanics and build the programs; 3) utilize the game mechanics during executing programs, thereby altering the state of the game environment, which provides feedback based on the constitutive rules; 4) continuously debugging according to the state of the game environment until achieving the goals.

Game flow theory posits that the optimal player experience is achieved when there is a harmonious balance between the game challenges and the player skills [29]. In the context of programming games, players engage in algorithmic problem-solving, which progressively introduces abstract computational concepts. The acquisition of such knowledge, along with the development of algorithmic construction skills, can be considered as the player skills. To sustain the flow state, programming games must continuously escalate the difficulty of algorithmic challenges, thereby promoting the acquisition of new computational concepts and enhancing the player's ability to solve increasingly complex problems. This dynamic interplay between challenge and skill not only ensures a

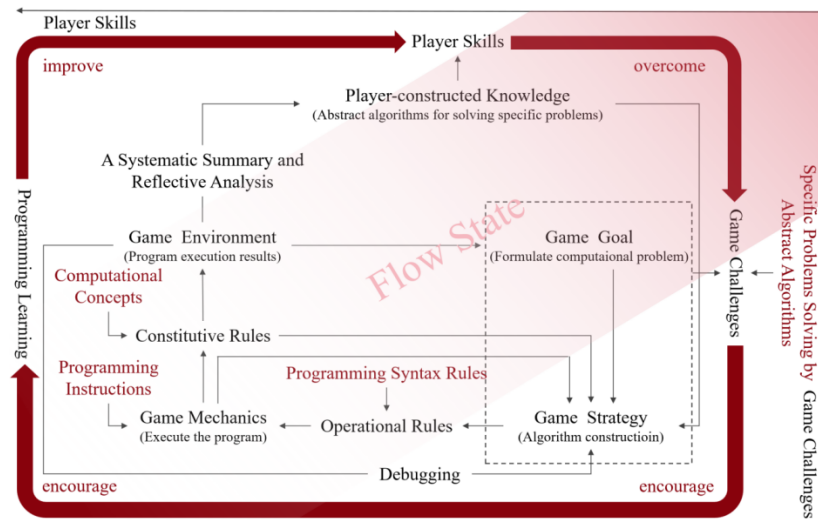


Figure 1: The framework for core elements in programming games

sustained level of engagement but also supports the development of CT. The framework is illustrated in Figure 1.

3.2 The Design of Core Elements in Programming Game WannaBone

Based on the framework, we designed a programming game called *WannaBone*. The game consists of five modules, totaling 20 levels, covering four fundamental computational concepts: sequences, loops (for loops and while loops), conditionals, and public boolean variables. Module 1 (Level 1~4) focuses on sequences, Module 2 (Level 5-7) centers on for loops, Module 3 (Level 8-10) emphasizes while loops, Module 4 (Level 11-14) addresses conditionals, and Module 5 (Level 15-20) explores public boolean variables. Players control two characters, “Big Dog” and “Puppy” through programming, while other uncontrollable game characters include monsters, landmines, altars, monster nest, and monster portal. Players need to program the big dog and puppy to eat meaty bone and pacifier without being harmed by monsters, landmines, and altars. The link to the game video is: <https://youtu.be/QtykBEsdB40>. The design of game mechanics, game constitutive rules, and game goals is detailed in the Table 1.

The operational rules of programming games dictate how players write programs within the game. Compared to code-based programming languages, visual programming languages significantly reduce the learning costs associated with programming syntax, allowing players to focus on algorithm construction. Therefore, *WannaBone* uses a combination of menus and buttons in its visual programming language, enabling students to build algorithms through simple clicks. The operational rules of the game are illustrated in Figure 2.

The challenges in programming games consist of puzzle-solving challenges and programming challenges. To effectively balance the educational objectives and gameplay, programming games need to organically integrate these two types of challenges—design puzzles based on the constitutive rules of computational concepts,

encouraging players to naturally understand and apply knowledge of computational concepts while thinking about how to solve the puzzles.

3.2.1 Game Challenges Design for “Loops”. Loops involve repeating instructions under specific conditions, such as for loops (which repeat a fixed number of times) and while loops (which repeat until a condition is met). In *WannaBone*, loops are integrated through puzzles like the altar, which hides the meaty bone. For example, in Level 5, the altar requires the player to sacrifice six monsters by using a for loop to repeat the “bark” action six times. In Level 8, the number of monsters is unknown, prompting the use of a while loop to have the big dog move and bark at specific locations until the bone appears. This design effectively integrates the concept of loops into the puzzle, as shown in Figure 3.

3.2.2 Game Challenges Design for “Conditionals”. Conditionals involve executing instructions based on specific conditions. In *WannaBone*, this concept is integrated through monster portals. For example, in Level 6, two portals are positioned in the upper and lower left corners, with one monster in the upper right corner and the other randomly moving between the portals. The game has two states: 1) the monster is in the upper portal, and 2) it is in the lower portal. The big dog can only bark when the monster is in the upper portal, luring both monsters downward to be eliminated by the landmine. If the dog barks while the monster is in the lower portal, it attacks the dog. In the puzzle-solving strategy, “when the monster is in the upper portal” becomes a specific condition, only when the game state meets this condition can the big dog bark. In this way, we integrate the concept of conditionals into a concrete puzzle, as shown in Figure 4.

3.2.3 Game Challenges Design for “Public Boolean Variables”. Public boolean variables allow different objects to access and modify shared data. To incorporate this concept into the puzzle, we designed two characters—big dog and puppy—who must cooperate

Table 1: The design of game mechanics, game constitutive rules, and game goals in *WannaBone*

Core Elements in <i>WannaBone</i>		Detailed Design for Game Core Elements
Game Mechanics	Big dog	Move up/down/left/right. Bark to lure awake monsters or to wake sleeping monsters.
	Puppy	Move up/down/left/right. Daze awake monsters.
	Loops	For loops: Repeat the actions a specified number of times. While loops: Repeat certain actions until the big dog eats the bone, until the puppy gets the pacifier, and until the meaty bone appears at the altar.
	Conditionals	Check if a certain monster is at a specific portal. Check if a certain signal is on or off.
	Public boolean variables	Turn a certain signal on or off. Get the switch status of a certain signal.
Game Constitutive Rules	Monster	Sleeping Monster: Non-aggressive; it becomes awake when it hears the big dog's bark. Awake Monster: Moves directly toward the position of the big dog. If the big dog enters its attack range, it will attack. If the puppy comes within range, it will be dazed and non-aggressive.
	Landmine	Attack big dog, puppy, or monsters when they enter attack range.
	Altar	Hide the meaty bone; it appears only after a sufficient number of creatures have been sacrificed.
	Monster nest	Monsters continuously surge in a certain direction.
	Monster portal	Monsters will randomly traverse among different portals.
Game Goals		From Level 1 to Level 14, the goal is for the "Big dog to eat the meaty bone". With the introduction of the puppy in Level 15, the goals expand to include both "Big dog eats the meaty bone" and "Puppy eats the pacifier".

in certain levels. For example, in Level 16, three monsters are positioned to the right of a landmine, and one is on the left. The player must first have the puppy daze the left-side monster, triggering a signal (setting a boolean variable to true). The big dog then continuously checks the signal before barking and moving towards the bone. If the signal is active (if the boolean variable is true), the big dog barks to lure the monsters, causing them to move left and be eliminated by the landmine. This level directly integrates the concept of public boolean variables within the puzzle, as reflected in Figure 5.

4 EXPERIMENTS AND RESULTS

4.1 Method

To assess the effectiveness of *WannaBone* in developing CT, we utilized the test developed by Roman-Gonzalez et al., which comprises 28 items covering sequences, loops, conditionals, and functions [30]. Given that *WannaBone* excludes functions but includes public variables, we modified the test by removing function-related items and incorporating relevant items on public variables from Code.org. We split the original test into two, each consisting of 18 items, administered before and after students engage with the game. The tests assess: simple structures (sequential and for loops), moderate structures (for loops and while loops), and complex structures (while loops, conditionals and public variables), with each structure containing 6 items.

To assess the effectiveness of *WannaBone* in enhancing learning motivation, we employ the validated scale proposed by Tuan et al., which has widespread application in academia and consists of 35 items covering 6 modules [31].

To evaluate the effectiveness of *WannaBone* in creating flow experience, grounded in the flow theory proposed by Sweetser et al. [29], a flow assessment scale was utilized, employing a five-point Likert format and comprises 28 items across 9 modules [32].

4.2 Procedure

The formal experiment involved 137 elementary school students from the 4th and 5th grades, and after the experiment, four assessment scales were collected from each participant. If any scale was incomplete, the completion time was too short (e.g., less than 25 seconds for the flow scale or less than 30 seconds for the learning motivation scale), or if participants provided consistent answers to contradictory items, all scales from that participant were deemed invalid. Ultimately, 22 participants were excluded, leaving 115 for data analysis, which included 60 males and 55 females.

4.3 Results

4.3.1 The Effect of *WannaBone* on Developing Computational Thinking. This section examines whether significant changes occur in participants' CT levels following the *WannaBone* game intervention. Given the skewed distribution identified by the Kolmogorov-Smirnov test, a paired samples Wilcoxon signed-rank test was utilized for data analysis. The results indicated a significant difference between the total scores of CT in the pre-test and post-test ($p\text{-value} = 0.000 < 0.01^{**}$), with post-test scores ($AVG = 8.01$, $SD = 2.415$) significantly exceeding pre-test scores ($AVG = 12.27$, $SD = 2.330$). The largest improvement was observed in complex structure, followed by moderate structure, while changes in simple structure were minimal as Table 2.

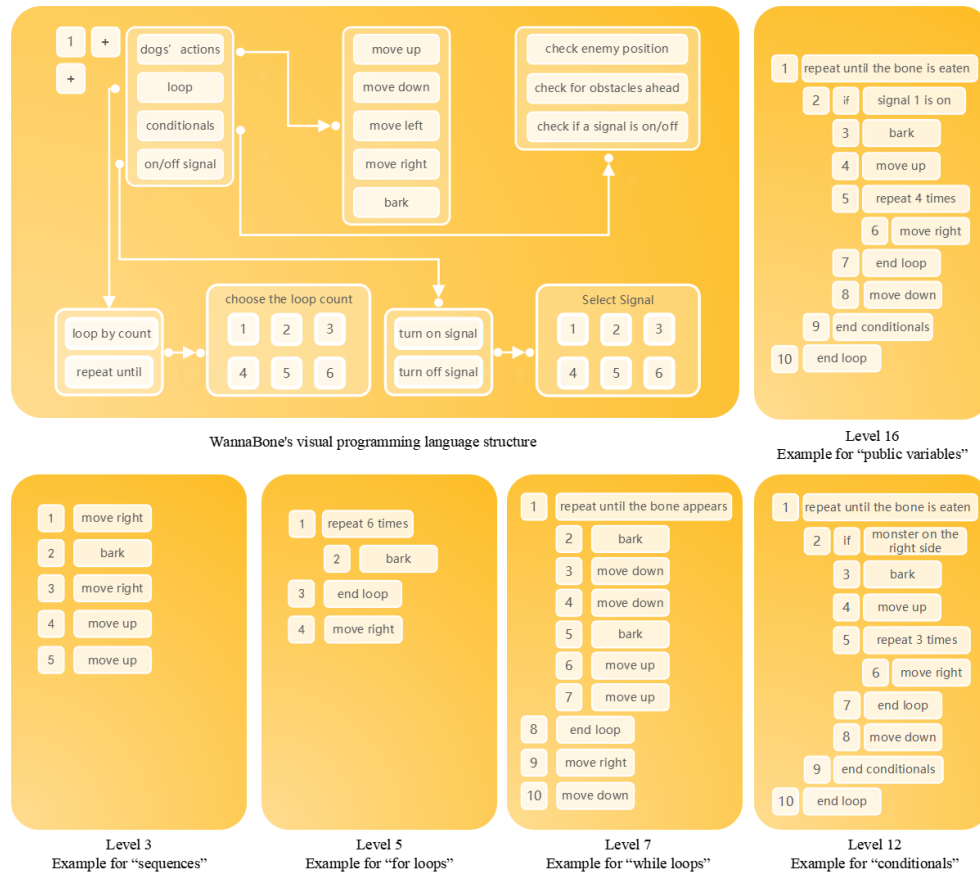
Figure 2: The design of operational rules in *WannaBone*

Figure 3: The game challenges design for "for loops" in Level 5 and for "while loops" in Level 8

Table 2: Paired samples Wilcoxon signed-rank test of CT scores in the pre-test and post-test

CT Level	Descriptive Statistics						Test Statistics	
	Pre-test			Post-test			Z	p
	AVG	SD	M	AVG	SD	M		
Simple Structure	3.86	1.263	4.00	4.34	1.337	5.00	-2.517	0.012**
Moderate Structure	2.55	1.488	3.00	4.10	1.294	4.00	-6.287	0.000**
Complex Structure	1.60	0.916	2.00	3.83	1.002	4.00	-8.853	0.000**

* $p < 0.05$ ** $p < 0.01$

Table 3: One-sample Wilcoxon signed-rank test analysis of learning motivation level in the pre-test and post-test

Learning Motivation	OM	Compared to GLM			Compared to SLM		
		HM	Z	p	HM	Z	p
Self Efficacy	-1.00	-9.00	7.762	0.000**	-2.00	-.598	0.550
Active Learning Strategies	33.00	24.00	7.330	0.000**	32.00	-.385	0.701
Science Learning Value	12.00	9.00	6.677	0.000**	12.00	-1.634	0.102
Performance Goal	-10.00	-12.00	4.057	0.000**	-8.00	-3.967	0.000**
Achievement Goal	20.00	15.00	6.754	0.000**	20.00	-1.301	0.193
Learning Environment Stimulation	19.00	15.00	6.453	0.000**	20.00	-1.997	0.046*

* p < 0.05 ** p < 0.01

Table 4: One-sample Wilcoxon signed-rank test of flow experience in the pre-test and post-test

Flow Experience	OM	Compared to GI			Compared to SI		
		HM	Z	p	HM	Z	p
Clear Goals	14.00	9.00	7.775	0.000**	12.00	2.379	0.017*
Unambiguous Feedback	13.00	9.00	7.356	0.000**	12.00	1.212	0.226
Challenge-skill Balance	16.00	12.00	6.536	0.000**	16.00	-.320	0.749
Concentration	13.00	9.00	7.008	0.000**	12.00	0.209	0.834
Action-awareness Merging	9.00	9.00	1.820	0.069	12.00	-4.775	0.000**
Sense of Control	12.00	9.00	5.623	0.000**	12.00	-1.295	0.195
Loss of self-consciousness	6.00	9.00	-3.657	0.000**	12.00	-7.964	0.000**
Transformation of Time	6.00	9.00	-2.990	0.003**	12.00	-7.525	0.000**
Autotelic Experience	12.00	9.00	5.776	0.000**	12.00	-1.677	0.094

* p < 0.05 ** p < 0.01

**Figure 4: The game challenge design for “conditionals” in Level 11****Figure 5: The game challenge design for “public boolean variables” in Level 16**

4.3.2 The Effect of WannaBone on Enhancing Learning Motivation. This section examines whether WannaBone effectively enhances participants' learning motivation. Cronbach's alpha of the scale is 0.942, the data for the six dimensions are 0.817, 0.945, 0.935, 0.895, 0.952, and 0.933, respectively. Since all alpha values are greater than 0.7, this indicates that the scale has a high level of reliability. The results of the KMO and Bartlett's test indicated that the data is suitable for factor analysis (KMO value = 0.902, p-value = 0.000 < 0.05). Items 2, 3, 17, and 18 showed weak validity and were excluded from the analysis.

Given the skewed distribution of data across the six dimensions of learning motivation, the one-sample Wilcoxon signed-rank test was employed. For each dimension of learning motivation, two hypothesized medians (HM) were tested: 1) "general learning motivation" (GLM), assuming that each participant selects "neutral" for all the items; 2) "strong learning motivation" (SLM), assuming that each participant selects "agree" for all the items (with reverse items receiving "disagree").

The results indicate that WannaBone effectively enhances participants' learning motivation. For each of the six dimensions, the observed median (OM) is significantly higher than the hypothesized



Figure 6: The testing of *WannaBone*

median for "general learning motivation" ($p\text{-value} < 0.01^{**}$). For the dimensions of "self-efficacy", "active learning strategies", "science learning value", and "achievement goals", the differences between the observed median and the hypothesized median for "strong learning motivation" in each dimension are not statistically significant ($p\text{-value} > 0.05^{*}$), indicating that participants' motivations in these dimensions are at a "strong" level. Conversely, for the dimensions of "performance goals" and "learning environment stimulation", significant differences are noted between the observed median and the hypothesized median for "strong learning motivation" in each dimension ($p\text{-value} < 0.05^{*}$), suggesting that participants' motivation lies between "general" and "strong" in these two dimensions. The results are shown in Table 4.

4.3.3 The Effect of *WannaBone* on Creating Flow Experience. This section aims to examine whether *WannaBone* can create a positive gaming experience that immerses participants. Cronbach's alpha of the data is 0.965, the alpha values for the nine dimensions are 0.942, 0.910, 0.963, 0.961, 0.941, 0.962, 0.960, 0.963, and 0.910, respectively. This indicates that the data from the scale have a high level of reliability. The results of the KMO and Bartlett's test indicated that the data is suitable for factor analysis (KMO value = 0.918, $p\text{-value} = 0.000 < 0.05$). Items 11 and 26 showed weak validity and were excluded from the analysis.

Since the data for the nine dimensions of flow experience do not conform to a normal distribution, the one-sample Wilcoxon signed-rank test was employed. For each dimension, two hypothesized medians were tested: 1) "general immersion" (GI), assuming that each participant selects "neutral" for all the items; 2) "strong immersion" (SI), assuming that each participant selects "agree" for all the items.

The dimensions of "clear goals", "unambiguous feedback", "challenge-skill balance", and "sense of control" are related to the core game elements. The results indicate that the observed medians for these dimensions are all significantly higher than their respective hypothesized medians for "general immersion" ($p\text{-value} = 0.000 < 0.01^{**}$). The results suggest that *WannaBone* provides timely feedback, appropriately game challenge, and a strong sense of control.

The dimensions of "concentration", "action-awareness merging", "loss of self-consciousness", and "transformation of time" are related to participants' psychological states. The observed median for "concentration" shows no significant difference from the hypothesized median for "strong immersion" ($p\text{-value} = 1.000 > 0.05$). This indicates that participants are able to maintain a high level of concentration. The observed median for "action-awareness merging" equals the hypothesized median for "general immersion" and shows no significant difference from it ($p\text{-value} = 0.069 > 0.05$), suggesting that the degree of merging between action and awareness is at a "general" level. The observed medians for "loss of self-consciousness" and "transformation of time" are both significantly below their respective "general immersion" hypothesized medians, indicating that participants do not lose their self-awareness or awareness of time passing. Nevertheless, the observed median for the "autotelic experience" is significantly higher than the "general immersion" hypothesized median ($p\text{-value} = 0.000 < 0.01^{**}$). This indicates that participants still feel a strong sense of pleasure and satisfaction. The results are reflected in Table 4.

5 DISCUSSION AND CONCLUSION

In this study, we proposed a framework for the core elements in programming games, reflects the design patterns of game mechanics, game challenges, game rules, and game goals, reveals how to integrate programming process and gaming process. Based on this framework, we developed the *WannaBone*. Through the experiments, we found that *WannaBone* effectively develop CT and creates a positive gaming experience, enhancing learning motivation. These results demonstrate the effectiveness of the framework. A limitation of this study is that *WannaBone* currently covers relatively few computational concepts. In the future, we will incorporate more computational concepts into the game and employ more diverse empirical research methods to further explore the impact of different design patterns on programming education outcomes.

Acknowledgments

This work is supported by Beijing Educational Science "14th Five-Year Plan" Project: "Research on the Model for Developing Computational Thinking in Primary School Students Based on Educational Programming Games"(Project Approval Number: CGA23109)

References

- [1] Charalampos Karagiannidis, Panagiotis Politis, and Ilias Karasavvidis. 2014. Research on E-Learning and ICT in Education. Springer, New York, USA.
- [2] Dan Zhao, Cristina H. Muntean, Adriana E. Chis, Gregor Rozinaj, and Gabriel-Miro Muntean. 2022. Game-Based Learning: Enhancing Student Experience, Knowledge Gain, and Usability in Higher Education Programming Courses. *IEEE Transactions on Education*. 65, 4 (November 2022), 502-513. <https://doi.org/10.1109/TE.2021.3136914>
- [3] Fwa H. Leong. 2016. Fine-Grained Detection of Programming Students' Frustration Using Keystrokes, Mouse Clicks and Interaction Logs. *Open Journal of Social Sciences*. 4, 9 (September 2016), 9-18. <https://doi.org/10.4236/jss.2016.49002>
- [4] Selvarajah Mohanarajah. 2018. Increasing Intrinsic Motivation of Programming Students: Towards Fix and Play Educational Games. *Issues in Informing Science and Information Technology*. 15 (March 2018): 69-77. <https://doi.org/10.28945/3996>
- [5] Elizabeth Boyle, Thomas M. Connolly, and Thomas Hainey. 2011. The Role of Psychology in Understanding the Impact of Computer Games. *Entertainment Computing*. 2, 2 (January 2011), 69-74. <https://doi.org/10.1016/j.entcom.2010.12.002>

- [6] Thomas M. Connolly, Elizabeth A. Boyle, Ewan MacArthur, Thomas Hainey, and James M. Boyle. 2012. A Systematic Literature Review of Empirical Evidence on Computer Games and Serious Games. *Computers & Education*. 59, 2 (March 2012): 661-686. <https://doi.org/10.1016/j.compedu.2012.03.004>
- [7] Sylvester Arnab, Theodore Lim, Maira B. Carvalho, Francesco Bellotti, Sara D. Freitas, Sandy Louchart, Neil Suttie, Riccardo Berta, and Alessandro D. Gloria. 2015. Mapping Learning and Game Mechanics for Serious Games Analysis. *British Journal of Educational Technology*. 46, 2 (March 2015), 391-411. <https://doi.org/10.1111/bjet.12113>
- [8] Mahwish Shahid, Afifa Wajid, Kamran U. Haq, Imran Saleem, and Abdul H. Shujia. 2019. A Review of Gamification for Learning Programming Fundamental. In *Proceedings of 2019 International Conference on Innovative Computing*. IEEE, Lahore, Pakistan, 1-8. <https://doi.org/10.1109/ICIC48496.2019.8966685>
- [9] Ioannis Paliokas, Chistos Arapidis, and Michail Mpimpitsos. 2013. Game Based Early Programming Education: The More You Play, The More You Learn. In *Transactions on Edutainment IX, Lecture Notes in Computer Science*. Springer, Berlin, Heidelberg, 115-131. https://doi.org/10.1007/978-3-642-37042-7_7
- [10] Bill Rouna and Fabiano Dalpiaz. 2016. A Model-driven Framework for Educational Game Design. *Games and Learning Alliance: 4th International Conference, Lecture Notes in Computer Science*. Springer, Cham, 1-11. https://doi.org/10.1007/978-3-319-40216-1_1
- [11] Michael A. Miljanovic and Jeremy S. Bradbury. 2018. A Review of Serious Games for Programming. *Joint International Conference on Serious Games (JCSG 2018), Lecture Notes in Computer Science*. Springer, Cham, 204-216. https://doi.org/10.1007/978-3-030-02762-9_21
- [12] José M. Corral, Antón C. Balcells, Arturo M. Estévez, Gabriel J. Moreno, and María J. Ramos. 2014. A Game-based Approach to the Teaching of Object-oriented Programming Languages. *Computers & Education*. 73, (April 2014), 83-92. <https://doi.org/10.1016/j.compedu.2013.12.013>
- [13] Cagin Kazimoglu, Mary Kiernan, Liz Bacon, and Lachlan MacKinnon. 2011. Understanding Computational Thinking before Programming: Developing Guidelines for the Design of Games to Learn Introductory Programming through Game-play. *International Journal of Game-Based Learning (IJGBL)*. 2011, 1, 3 (September 2011), 30-52. <https://doi.org/10.4018/ijgb.2011070103>
- [14] Polina Charters, Michael J. Lee, Amy J. Ko, and Dastyni Loksa. 2014. Challenging Stereotypes and Changing Attitudes: The Effect of a Brief Programming Encounter on Adults' Attitudes toward Programming. In *Proceedings of the 45th ACM Technical Symposium on Computer Science Education*. ACM, Atlanta, GA, 653-658. <http://dx.doi.org/10.1145/2538862.2538938>
- [15] Anuradha Mathrani, Shelly Christian, and Agate Ponder-Sutton. PlayIT: Game Based Learning Approach for Teaching Programming Concepts. *Journal of Educational Technology & Society*. 19, 2 (April 2016), 5-17.
- [16] Ernest Adams. 2009. *Fundamentals of Game Design* (2 edition.). New Riders. Berkeley, USA.
- [17] Kevin Werbach and Dan Hunter. 2012. *For the Win: How Game Thinking Can Revolutionize Your Business*. Wharton School Press. Philadelphia, USA.
- [18] Jesper Juul. 2003. *The Game, the Player, the World: Looking for a Heart of Gamesness*. In *Proceedings of DIGRA 2003 Conference: Level Up*. Utrecht University and DiGRA, Utrecht, Netherlands, 30-45.
- [19] Michael S. Debus. 2020. *Unifying Game Ontology: A Faceted Classification of Game Elements*. PhD Thesis, IT University of Copenhagen.
- [20] Martin Sillaots, Triinu Jesmin, and Andrus Rinde. 2016. Survey for Mapping Game Elements. In *Proceedings of the 10th European Conference on Games Based Learning*. Academic Conferences International(ACI), Paisley, UK, 617-626.
- [21] Katie S. Tekinbas and Eric Zimmerman. 2003. *Rules of Play: Game Design Fundamental*. The MIT Press. Massachusetts, UK.
- [22] Miguel Sicart. 2008. Defining Game Mechanics. *Game Studies*. 8, 2 (December 2008), 1-15.
- [23] Aki Järvinen. 2008. *Games without Frontiers: Theories and Methods for Game Studies and Design*. Tampere University Press. Tampere, Finland.
- [24] Olga Shabalina, Pavel Vorobkalov, Alexander Kataev, and Alexey Tarasenko. 2008. Educational Games for Learning Programming Languages. In *Proceedings of the Third International Conference "Modern (e-) Learning"*. FOI ITHEA, Varna, Bulgaria, 79-83.
- [25] Christos Malliarakis, Maya Satratzemi, and Stelios Xinogalos. 2016. CMX: The Effects of an Educational MMORPG on Learning and Teaching Computer Programming. *IEEE Transactions on Learning Technologies*. 10, 2 (April 2016), 219-235. <https://doi.org/10.1109/TLT.2016.2556666>
- [26] Xina Jiang, Casper Hartevel, Yubin Yang, Anthony Fung, Xinyuan Huang, and Shihong Chen. 2023. "If it's sunny, don't take an umbrella": a systematic evaluation of design principles for CT teaching games. *Educational technology research and development*. 71, 4 (May 2023), 1725-1763. <https://doi.org/10.1007/s11423-023-10240-1>
- [27] Chen-Chung Liu, Yuan-Bang Cheng, and Chia-Wen Huang. The Effect of Simulation Games on the Learning of Computational Problem Solving. *Computers & Education*. 57, 3 (November 2011), 1907-1918. <https://doi.org/10.1016/j.compedu.2011.04.002>
- [28] Xina Jiang, Casper Hartevel, Xinyuan Huang, and Anthony Y. Fung. 2019. The Computational Puzzle Design Framework: A Design Guide for Games Teaching Computational Thinking. In *Proceedings of the 14th International Conference on the Foundations of Digital Games (FDG '19)*. ACM, San Luis Obispo, CA, 1-11. <https://doi.org/10.1145/3337722.3337768>
- [29] Penelope Sweetser and Peta Wyeth. 2005. GameFlow: a model for evaluating player enjoyment in games. *Computers in Entertainment (CIE)*. 3, 3 (July 2005), 1-24. <https://doi.org/10.1145/1077246.1077253>
- [30] Marcos Román-González, Juan-Carlos Pérez-González, and Carmen Jiménez-Fernández. 2017. Which cognitive abilities underlie computational thinking? Criterion validity of the Computational Thinking Test. *Computers in human behavior*. 72, (July 2017), 678-691. <https://doi.org/10.1016/j.chb.2016.08.047>
- [31] Hsiao L. Tuan, Chi C. Chin, and Shyang H. Shieh. 2005. The development of a questionnaire to measure students' motivation towards science learning. *International journal of science education*. 27, 6 (May 2011), 639-654. <https://doi.org/10.1080/0950069042000323737>
- [32] Xiaowei Cai, Javier Cebollada, and Mónica Cortiñas. 2022. Self-report measure of dispositional flow experience in the video game context: Conceptualisation and scale development. *International Journal of Human-Computer Studies*. 159, (March 2022), 102746. <https://doi.org/10.1016/j.ijhcs.2021.102746>